ELASTICSEARCH WORKSHOP

Category 7: Aggregations — Bucketing

EXERCISES

Scenario

You are a data engineer for a fictional intelligence unit called UNIT-7.
Your unit logs every border crossing incident at checkpoints across four sectors.
You will use bucketing aggregations to group and count these incidents.
All names, locations, and incident codes are completely fictional.

Key Concept: Bucketing Aggregations

A bucketing aggregation groups documents into buckets. Each bucket has a key and a count. Unlike metric aggregations which return one number, bucketing returns a list of groups.

Aggregation	What it does
`terms`	Groups documents by the unique values in a keyword field. Like SQL GROUP BY.
`range`	Groups documents into numeric ranges you define. Example: 0-10, 10-50, 50+.
`date_histogram`	Groups documents by time period. Example: one bucket per month.

You can also nest a metric aggregation inside a bucket. This gives you stats per group. Example: average response_time per checkpoint. This is called a sub-aggregation.

Structure of a bucketing aggregation:

```
{
    'size': 0,
    'aggs': {
        'by_sector': {
            'terms': { 'field': 'sector' }
        }
    }
}
```

Result is in: response['aggregations']['by_sector']['buckets']
Each bucket has: 'key' (the group value) and 'doc_count' (how many docs).

SQL analogy: SELECT sector, COUNT(*) FROM incidents GROUP BY sector

Dataset: Border Crossing Incident Log

Index name: 'incidents' | 20 records

incident_id	checkpoint	sector	incident_type	severity	suspects	response_min	incident_date	resolved
INC-001	CP-NORTH-1	NORTH	smuggling	HIGH	3	22	2024-01-04	true
INC-002	CP-SOUTH-1	SOUTH	infiltration	HIGH	2	35	2024-01-08	true
INC-003	CP-EAST-1	EAST	smuggling	MEDIUM	1	18	2024-01-12	true
INC-004	CP-NORTH-2	NORTH	trespassing	LOW	1	10	2024-01-17	true
INC-005	CP-WEST-1	WEST	infiltration	HIGH	4	48	2024-01-22	false
INC-006	CP-SOUTH-2	SOUTH	smuggling	MEDIUM	2	25	2024-01-28	true
INC-007	CP-EAST-1	EAST	trespassing	LOW	1	8	2024-02-03	true
INC-008	CP-NORTH-1	NORTH	infiltration	HIGH	3	40	2024-02-08	true
INC-009	CP-WEST-1	WEST	smuggling	HIGH	5	55	2024-02-13	false
INC-010	CP-SOUTH-1	SOUTH	trespassing	LOW	1	12	2024-02-18	true
INC-011	CP-EAST-2	EAST	smuggling	MEDIUM	2	20	2024-02-23	true
INC-012	CP-NORTH-2	NORTH	smuggling	MEDIUM	2	28	2024-03-01	true
INC-013	CP-WEST-1	WEST	infiltration	HIGH	3	42	2024-03-05	true
INC-014	CP-SOUTH-2	SOUTH	smuggling	HIGH	4	38	2024-03-09	false
INC-015	CP-EAST-1	EAST	infiltration	MEDIUM	2	30	2024-03-13	true
INC-016	CP-NORTH-1	NORTH	trespassing	LOW	1	9	2024-03-17	true
INC-017	CP-WEST-1	WEST	smuggling	MEDIUM	2	22	2024-03-21	true
INC-018	CP-SOUTH-1	SOUTH	infiltration	HIGH	3	45	2024-03-25	false
INC-019	CP-EAST-2	EAST	trespassing	LOW	1	11	2024-03-28	true
INC-020	CP-NORTH-2	NORTH	smuggling	HIGH	4	33	2024-03-31	true

response_min = response time in minutes. suspects = number of suspects involved.

Exercise 1 [Step-by-step]

Task: Create the 'incidents' index with the correct mapping. Bulk index all 20 records.

Step 1 — Create the index

```
from elasticsearch import Elasticsearch

es = Elasticsearch('http://localhost:9200')

if es.indices.exists(index='incidents'):
    es.indices.delete(index='incidents')

mapping = {
    'mappings': { 'properties': {
        'incident_id':   { 'type': 'keyword' },
        'checkpoint':    { 'type': 'keyword' },
        'sector':        { 'type': 'keyword' },
        'incident_type': { 'type': 'keyword' },
        'severity':      { 'type': 'keyword' },
        'suspects':      { 'type': 'integer' },
        'response_min':  { 'type': 'integer' },
        'incident_date': { 'type': 'date'    },
        'resolved':      { 'type': 'boolean' }
    }}
}
es.indices.create(index='incidents', body=mapping)
print('Index created.')
```

Step 2 — Bulk index all 20 incidents

```
from elasticsearch.helpers import bulk

incidents = [

{'_index':'incidents','_id':'INC-001','_source':{'incident_id':'INC-001','checkpoin
t':'CP-NORTH-1','sector':'NORTH','incident_type':'smuggling','severity':'HIGH','sus
pects':3,'response_min':22,'incident_date':'2024-01-04','resolved':True}},

{'_index':'incidents','_id':'INC-002','_source':{'incident_id':'INC-002','checkpoin
t':'CP-SOUTH-1','sector':'SOUTH','incident_type':'infiltration','severity':'HIGH','
suspects':2,'response_min':35,'incident_date':'2024-01-08','resolved':True}},

{'_index':'incidents','_id':'INC-003','_source':{'incident_id':'INC-003','checkpoin
t':'CP-EAST-1','sector':'EAST','incident_type':'smuggling','severity':'MEDIUM','sus
pects':1,'response_min':18,'incident_date':'2024-01-12','resolved':True}},

{'_index':'incidents','_id':'INC-004','_source':{'incident_id':'INC-004','checkpoin
t':'CP-NORTH-2','sector':'NORTH','incident_type':'trespassing','severity':'LOW','su
spects':1,'response_min':10,'incident_date':'2024-01-17','resolved':True}},
```

{'_index':'incidents','_id':'INC-005','_source':{'incident_id':'INC-005','checkpoint':'CP-WEST-1','sector':'WEST','incident_type':'infiltration','severity':'HIGH','suspects':4,'response_min':48,'incident_date':'2024-01-22','resolved':False}},

{'_index':'incidents','_id':'INC-006','_source':{'incident_id':'INC-006','checkpoint':'CP-SOUTH-2','sector':'SOUTH','incident_type':'smuggling','severity':'MEDIUM','suspects':2,'response_min':25,'incident_date':'2024-01-28','resolved':True}},

{'_index':'incidents','_id':'INC-007','_source':{'incident_id':'INC-007','checkpoint':'CP-EAST-1','sector':'EAST','incident_type':'trespassing','severity':'LOW','suspects':1,'response_min':8,'incident_date':'2024-02-03','resolved':True}},

{'_index':'incidents','_id':'INC-008','_source':{'incident_id':'INC-008','checkpoint':'CP-NORTH-1','sector':'NORTH','incident_type':'infiltration','severity':'HIGH','suspects':3,'response_min':40,'incident_date':'2024-02-08','resolved':True}},

{'_index':'incidents','_id':'INC-009','_source':{'incident_id':'INC-009','checkpoint':'CP-WEST-1','sector':'WEST','incident_type':'smuggling','severity':'HIGH','suspects':5,'response_min':55,'incident_date':'2024-02-13','resolved':False}},

{'_index':'incidents','_id':'INC-010','_source':{'incident_id':'INC-010','checkpoint':'CP-SOUTH-1','sector':'SOUTH','incident_type':'trespassing','severity':'LOW','suspects':1,'response_min':12,'incident_date':'2024-02-18','resolved':True}},

{'_index':'incidents','_id':'INC-011','_source':{'incident_id':'INC-011','checkpoint':'CP-EAST-2','sector':'EAST','incident_type':'smuggling','severity':'MEDIUM','suspects':2,'response_min':20,'incident_date':'2024-02-23','resolved':True}},

{'_index':'incidents','_id':'INC-012','_source':{'incident_id':'INC-012','checkpoint':'CP-NORTH-2','sector':'NORTH','incident_type':'smuggling','severity':'MEDIUM','suspects':2,'response_min':28,'incident_date':'2024-03-01','resolved':True}},

{'_index':'incidents','_id':'INC-013','_source':{'incident_id':'INC-013','checkpoint':'CP-WEST-1','sector':'WEST','incident_type':'infiltration','severity':'HIGH','suspects':3,'response_min':42,'incident_date':'2024-03-05','resolved':True}},

{'_index':'incidents','_id':'INC-014','_source':{'incident_id':'INC-014','checkpoint':'CP-SOUTH-2','sector':'SOUTH','incident_type':'smuggling','severity':'HIGH','suspects':4,'response_min':38,'incident_date':'2024-03-09','resolved':False}},

{'_index':'incidents','_id':'INC-015','_source':{'incident_id':'INC-015','checkpoint':'CP-EAST-1','sector':'EAST','incident_type':'infiltration','severity':'MEDIUM','suspects':2,'response_min':30,'incident_date':'2024-03-13','resolved':True}},

{'_index':'incidents','_id':'INC-016','_source':{'incident_id':'INC-016','checkpoint':'CP-NORTH-1','sector':'NORTH','incident_type':'trespassing','severity':'LOW','suspects':1,'response_min':9,'incident_date':'2024-03-17','resolved':True}},

{'_index':'incidents','_id':'INC-017','_source':{'incident_id':'INC-017','checkpoint':'CP-WEST-1','sector':'WEST','incident_type':'smuggling','severity':'MEDIUM','suspects':2,'response_min':22,'incident_date':'2024-03-21','resolved':True}},

{'_index':'incidents','_id':'INC-018','_source':{'incident_id':'INC-018','checkpoint':'CP-SOUTH-1','sector':'SOUTH','incident_type':'infiltration','severity':'HIGH','suspects':3,'response_min':45,'incident_date':'2024-03-25','resolved':False}},

{'_index':'incidents','_id':'INC-019','_source':{'incident_id':'INC-019','checkpoint':'CP-EAST-2','sector':'EAST','incident_type':'trespassing','severity':'LOW','suspects':1,'response_min':11,'incident_date':'2024-03-28','resolved':True}},

{'_index':'incidents','_id':'INC-020','_source':{'incident_id':'INC-020','checkpoint':'CP-NORTH-2','sector':'NORTH','incident_type':'smuggling','severity':'HIGH','suspects':4,'response_min':33,'incident_date':'2024-03-31','resolved':True}},

```
]
success, errors = bulk(es, incidents)
print('Indexed:', success, '| Errors:', errors)
```

Exercise 2 [Step-by-step] (required)

Task: Count how many incidents occurred in each sector.

Use a terms aggregation on the 'sector' field.
Print each sector and its incident count.

Step 1 — Write the terms aggregation

```
from elasticsearch import Elasticsearch

es = Elasticsearch('http://localhost:9200')

# terms aggregation: group documents by the unique values in a keyword field.
# 'field': 'sector' means: create one bucket for each unique sector value.
# Each bucket contains all documents with that sector value.
# 'size': 10 inside the terms means: return up to 10 buckets.
#   (This is different from the outer 'size' which controls document hits.)
query = {
    'size': 0,
    'aggs': {
        'by_sector': {
            'terms': {
                'field': 'sector',
                'size':  10
            }
        }
    }
}
```

Step 2 — Run and read the buckets

```
response = es.search(index='incidents', body=query)

# Bucketing results are in ['buckets']. It is a list.
# Each item is a bucket with 'key' (the sector name) and 'doc_count'.
buckets = response['aggregations']['by_sector']['buckets']

print('Incidents per sector:')
for bucket in buckets:
    print(' ', bucket['key'], ':', bucket['doc_count'])

# Expected output (sorted by count, highest first by default):
# NORTH : 6
```

```
# SOUTH : 5
# EAST  : 5
# WEST  : 4
```

```
Command reference:
  'terms': { 'field': 'keyword_field', 'size': N }
  response['aggregations']['name']['buckets']   List of buckets.
  bucket['key']        The group value (e.g. 'NORTH').
  bucket['doc_count']  How many documents are in this bucket.

  Default sort: highest doc_count first.
  'size' inside terms controls max number of buckets returned.

SQL analogy:     SELECT sector, COUNT(*) FROM incidents GROUP BY sector ORDER BY
COUNT(*) DESC
```

Exercise 3

Task: Count how many incidents of each incident_type occurred. Sort the output alphabetically by type name.

Use a terms aggregation on incident_type.
To sort alphabetically by the bucket key, add an 'order' option to the terms aggregation.

Print each incident_type and its count.

Expected result: infiltration=6, smuggling=9, trespassing=5.

```
Command reference:
  'terms': {
      'field': 'incident_type',
      'size':  10,
      'order': { '_key': 'asc' }
  }

  '_key': 'asc'    Sort buckets alphabetically by the key value.
  '_count': 'desc' Sort by doc_count highest first (this is the default).

SQL analogy:     SELECT incident_type, COUNT(*) FROM incidents GROUP BY incident_type
ORDER BY incident_type ASC
```

Exercise 4 (required)

Task: For each sector, count the incidents AND calculate the average response_min.

Use a terms aggregation on sector. Inside it, add a sub-aggregation for avg response_min.
A sub-aggregation runs inside each bucket. It calculates a metric per group.

Print each sector, its incident count, and its average response time.

Expected result: NORTH avg~28.0 min, SOUTH avg~30.8 min, EAST avg~17.4 min, WEST avg~37.8 min.

```
Command reference:
  'by_sector': {
      'terms': { 'field': 'sector', 'size': 10 },
      'aggs':  {
          'avg_response': { 'avg': { 'field': 'response_min' } }
      }
  }

  The inner 'aggs' is the sub-aggregation. It runs inside each bucket.
  Access with: bucket['avg_response']['value']

SQL analogy:     SELECT sector, COUNT(*), AVG(response_min) FROM incidents GROUP BY sector
```

Exercise 5

Task: Count unresolved incidents per sector.

Filter the query first: resolved = false. Then apply a terms aggregation on sector.
Only unresolved incidents should be counted in the buckets.

Expected result: WEST=2, SOUTH=2. NORTH and EAST have no unresolved incidents.

```
Command reference:
  Combine query filter and terms agg:
  {
      'size': 0,
      'query': { 'bool': { 'filter': [ { 'term': { 'resolved': False } } ] } },
```

```
    'aggs':  { 'unresolved_by_sector': { 'terms': { 'field': 'sector', 'size': 10 } } }
  }

SQL analogy:     SELECT sector, COUNT(*) FROM incidents WHERE resolved = false GROUP BY
sector
```

Exercise 6

Task: Group incidents by response time into three bands: fast (under 15 min), medium (15-35 min), slow (over 35 min).

Use a range aggregation on response_min. You define the bucket boundaries.
Print each band and how many incidents fall into it.

Expected result: fast (0-15)=5, medium (15-35)=10, slow (35+)=5.

```
Command reference:
  'response_bands': {
      'range': {
          'field': 'response_min',
          'ranges': [
              { 'key': 'fast',   'to': 15 },
              { 'key': 'medium', 'from': 15, 'to': 35 },
              { 'key': 'slow',   'from': 35 }
          ]
      }
  }

  'key' is an optional label for the bucket. Without it, ES uses 'from-to' as the key.
  'from' is inclusive (>=). 'to' is exclusive (<).
  A bucket with only 'to': 15 means: all values < 15.
  A bucket with only 'from': 35 means: all values >= 35.

SQL analogy:
  SELECT CASE WHEN response_min < 15 THEN 'fast'
              WHEN response_min < 35 THEN 'medium'
              ELSE 'slow' END, COUNT(*)
  FROM incidents GROUP BY 1
```

Exercise 7

Task: For each response time band (fast / medium / slow), also calculate the average number of suspects.

Add a sub-aggregation inside the range buckets. Calculate avg suspects per band.

Expected result: fast avg suspects~1.0, medium avg suspects~2.1, slow avg suspects~3.4.

```
Command reference:
 'response_bands': {
     'range': { 'field': 'response_min', 'ranges': [...] },
     'aggs':  { 'avg_suspects': { 'avg': { 'field': 'suspects' } } }
 }

 Sub-aggregations work the same inside range buckets as inside terms buckets.
 Access with: bucket['avg_suspects']['value']
```

Exercise 8 (required)

Task: Count how many incidents occurred each month.

Use a date_histogram aggregation on incident_date.
Set the calendar interval to 'month'. Each bucket will represent one month.

Print each month key and the incident count.

Expected result: 2024-01 = 6, 2024-02 = 5, 2024-03 = 9.

```
Command reference:
 'by_month': {
     'date_histogram': {
         'field':            'incident_date',
         'calendar_interval': 'month'
     }
 }

 calendar_interval values:
   minute, hour, day, week, month, quarter, year

 Each bucket key is the start of the time period as a timestamp.
 Use 'format' to display it as a readable string:
   'format': 'yyyy-MM'   shows '2024-01', '2024-02', etc.
```

```
SQL analogy:    SELECT DATE_FORMAT(incident_date,'%Y-%m'), COUNT(*) FROM incidents
GROUP BY 1
```

Exercise 9

Task: For each month, count incidents AND sum the total suspects involved.

Use a date_histogram on incident_date with calendar_interval 'month'.
Add a sub-aggregation for sum of suspects.

Expected result: Jan=6 incidents/13 suspects, Feb=5 incidents/12 suspects, Mar=9 incidents/20 suspects.

```
Command reference:
  'by_month': {
      'date_histogram': {
          'field':            'incident_date',
          'calendar_interval': 'month',
          'format':           'yyyy-MM'
      },
      'aggs': {
          'total_suspects': { 'sum': { 'field': 'suspects' } }
      }
  }

  Access: bucket['total_suspects']['value']
```

Exercise 10

Task: For each sector, count incidents broken down by incident_type.

Use a terms aggregation on sector. Inside it, add a sub-aggregation that is ALSO a terms aggregation on incident_type.
This is called nested bucketing. It gives you a count per sector per type.

Print: sector > incident_type > count for all combinations.

Expected result example: NORTH has smuggling=3, infiltration=1, trespassing=2.

```
Command reference:
  'by_sector': {
      'terms': { 'field': 'sector', 'size': 10 },
      'aggs': {
          'by_type': {
              'terms': { 'field': 'incident_type', 'size': 10 }
          }
      }
  }

  Accessing nested buckets:
  for sector_bucket in sector_buckets:
      for type_bucket in sector_bucket['by_type']['buckets']:
          print(sector_bucket['key'], '>', type_bucket['key'], ':', type_bucket['doc_count'])

SQL analogy:     SELECT sector, incident_type, COUNT(*) FROM incidents GROUP BY sector,
incident_type
```

End of Category 7 Exercises